

7. Satisfying Functional Requirements

To validate the correctness of the payment flow, we first design a simplified version of the system running on a single machine for a small number of users.

The goal at this stage is not scalability, but ensuring that all functional requirements are properly satisfied.

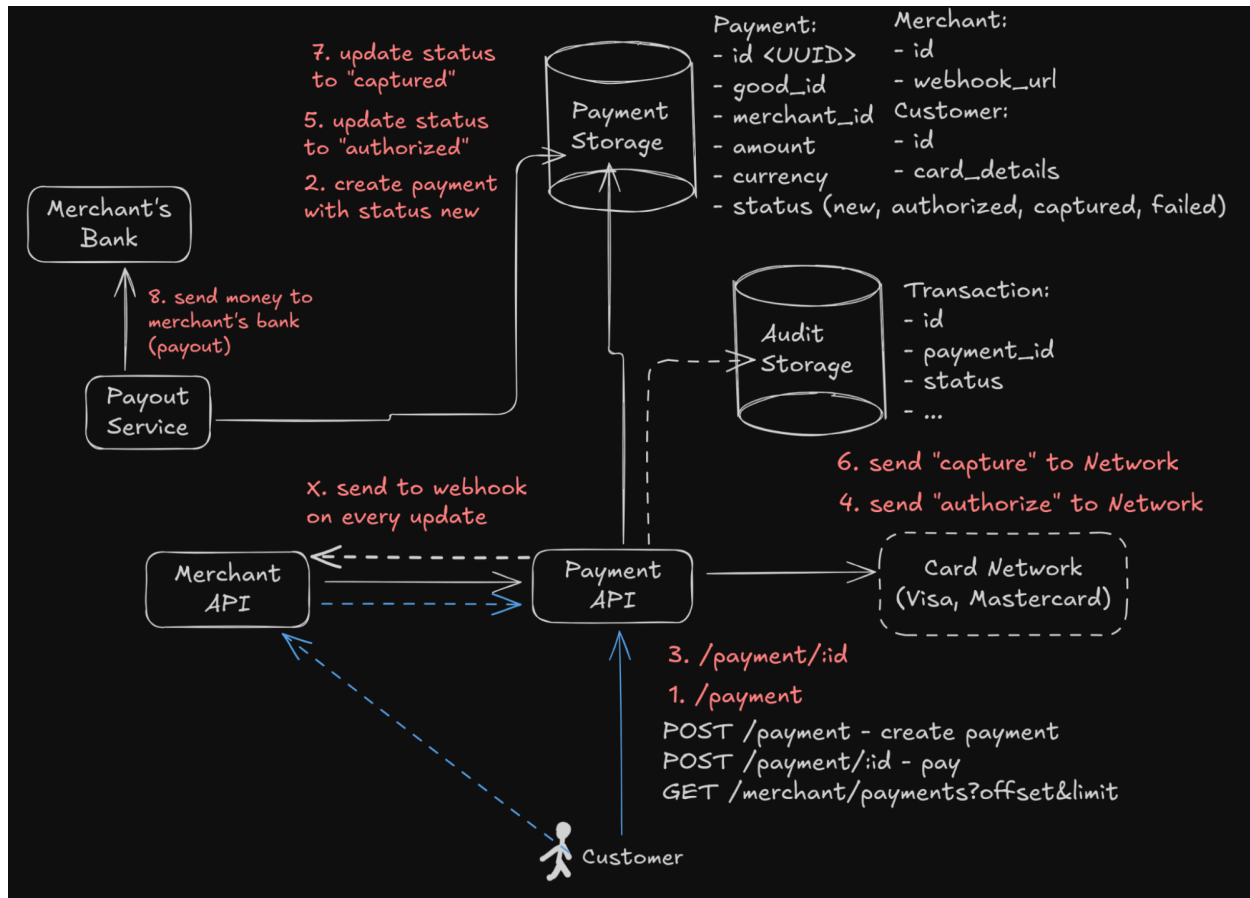
7.1 High-Level Architecture

The system consists of the following major components:

- Merchant API
- Payment API
- Payment Storage
- Audit Storage (Ledger)
- Card Network Integration
- Payout Service
- Merchant's Bank

The overall workflow is shown below.

Functional Design Diagram



7.2 Core Components

1. Merchant API

Responsible for merchant-facing operations:

- Merchant registration
- Webhook configuration
- Payment listing APIs

Example APIs:

None

`GET /merchant/payments?offset&limit`

2. Payment API

Main entry point for payment processing.

Responsibilities:

- Create payment intents
- Process authorization and capture
- Communicate with card networks
- Update payment status
- Trigger webhooks

Example APIs:

None

POST /payment

POST /payment/{id}

GET /payment/{id}

3. Payment Storage

Stores the current state of payments.

Payment Entity

Field	Description
id	Unique payment identifier (UUID)
good_id	Purchased item/service
merchant_id	Merchant reference

amount	Payment amount
currency	Currency code
status	new / authorized / captured / failed

Merchant Entity

Field	Description
id	Merchant identifier
webhook_url	Merchant callback URL

Customer Entity

Field	Description
id	Customer identifier
card_details	Card information

4. Audit Storage (Ledger)

Stores immutable financial transaction history.

Transaction Entity

Field	Description
id	Transaction identifier
payment_id	Associated payment
status	Transaction state
...	Additional metadata

Purpose:

- Auditability
- Financial traceability
- Legal compliance

5. Card Network Integration

External integration with:

- Visa
- Mastercard
- Other card processors

Responsibilities:

- Authorization requests
- Capture requests
- Settlement coordination

6. Payout Service

Responsible for transferring money:

- From Stripe's bank
- To merchant's bank account

This happens after successful payment capture and settlement.

7.3 Step-by-Step Payment Flow

Step 1 — Create Payment

Customer initiates payment through merchant.

Request:

None

POST /payment

Actions:

- Create payment record
- Status = NEW
- Store in Payment Storage

Step 2 — Persist Payment

Payment is saved in database with initial state.

Example:

JSON

```
{  
  
  "payment_id": "uuid",
```

```
"status": "NEW"
```

```
}
```

Step 3 — Retrieve Payment

Customer or merchant can fetch payment status.

Request:

None

```
GET /payment/{id}
```

Step 4 — Authorization Request

Payment API sends authorization request to Card Network.

Purpose:

- Validate card
- Check balance
- Perform fraud checks
- Reserve money

Possible results:

- Authorized
- Declined

Step 5 — Update Status to AUTHORIZED

If bank approves:

- Payment status becomes **AUTHORIZED**

Stored in:

- Payment Storage
- Audit Storage

Webhook may also be triggered.

Step 6 — Capture Request

Payment API sends capture request to Card Network.

Purpose:

- Confirm transaction
- Move reserved funds for settlement

Important:

- Capture operation must be idempotent

Step 7 — Update Status to CAPTURED

After successful capture:

- Payment status becomes CAPTURED

Actions:

- Create ledger entries
- Trigger merchant webhook
- Prepare payout flow

Step 8 — Payout to Merchant

Payout Service transfers funds:

- Stripe bank → Merchant bank

After payout:

- Ledger updated
- Merchant balance settled

7.4 Webhook Flow

On every important payment update:

- Webhook event is sent to merchant

Examples:

- payment.authorized
- payment.captured
- payment.failed

Delivery guarantee:

- At least once
- Retries on failure

7.5 Ensuring Functional Requirements

Requirement	Solution
Merchant registration	Merchant API
Webhook support	Webhook delivery service
Customer payments	Payment API

Merchant payouts	Payout Service
No double charge	Idempotent capture/payment requests
Audit compliance	Immutable Audit Storage

7.6 Important Design Observations

This simplified design proves:

- Payment lifecycle works correctly
- State transitions are manageable
- Ledger ensures auditability
- Payouts can be processed
- Webhooks notify merchants reliably

However, this architecture still has limitations:

- Single database bottleneck
- No fault tolerance
- No horizontal scaling
- Synchronous processing
- Cannot handle 10k+ RPS traffic

These issues will be addressed while satisfying the non-functional requirements in the distributed design.

7.7 Merchant Registration & Webhook Configuration

One of the first functional requirements is allowing merchants to onboard onto the platform and configure webhook endpoints for payment notifications.

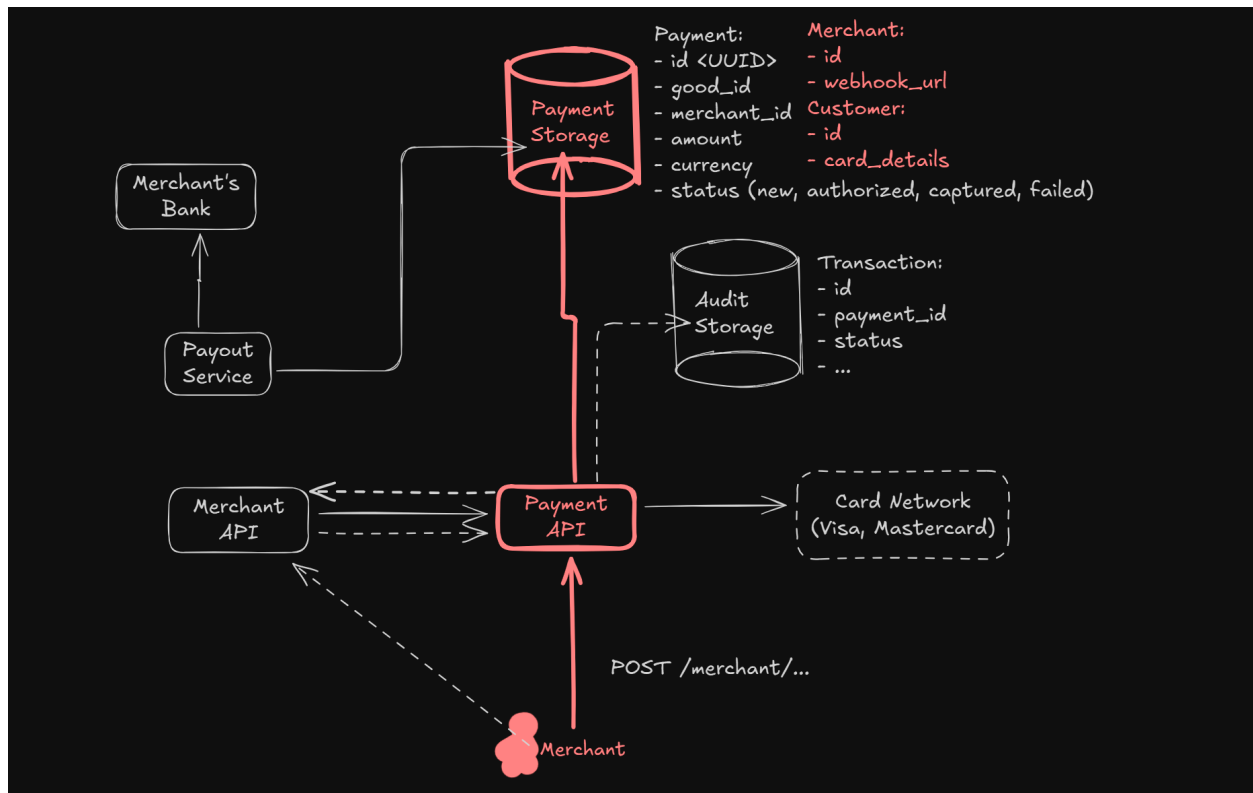
At a basic level, this can be implemented using:

- A simple API
- Persistent storage for merchant data

7.7.1 High-Level Design

The merchant interacts with the system through the **Merchant API**, which communicates with the Payment API and stores merchant-related information in persistent storage.

Merchant Registration Flow



7.7.2 Merchant Registration API

A merchant registers itself by sending merchant-related details to the system.

Example API:

None

POST /merchant

Example request payload:

JSON

```
{  
  "name": "Acme Store",  
  "webhook_url": "https://merchant.com/webhook",  
  "payment_methods": ["card"],  
  "logo_url": "https://merchant.com/logo.png"  
}
```

7.7.3 Merchant Data Model

The system stores merchant-related metadata in persistent storage.

Merchant Entity

Field	Description
id	Unique merchant identifier
name	Merchant name

webhook_url	Merchant callback URL
api_key	Public API key
api_secret	Secret key for authentication
payment_methods	Supported payment methods
logo_url	Merchant branding
created_at	Registration timestamp

7.7.4 Webhook Support

Merchants configure webhook URLs to receive asynchronous notifications about events such as:

- Payment authorized
- Payment captured
- Payment failed
- Payout completed

Example webhook event:

```
JSON
{
  "event": "payment.captured",
  "payment_id": "pay_123",
```

```
"amount": 100,  
"currency": "USD"  
}
```

7.7.5 API Security

Since payment systems are highly sensitive, APIs must be protected using authentication and signature validation mechanisms.

API Authentication

Each merchant receives:

- API Key
- API Secret

These are used to:

- Authenticate API requests
- Prevent unauthorized access

Example:

```
None  
Authorization: Bearer <api_key>
```

Webhook Signature Validation

Webhook requests must be signed to ensure:

- Authenticity

- Integrity
- Protection against spoofed requests

Flow

1. Stripe generates a signature using secret key
2. Signature is attached in webhook headers
3. Merchant validates signature before processing event

Example:

None

X-Stripe-Signature: abc123...

7.7.6 Real-World Merchant Data

In production systems, merchant storage contains much more information, including:

- Business details
- KYC/KYB documents
- Supported currencies
- Settlement bank accounts
- Fraud settings
- Tax information
- Regional configurations
- Branding assets

7.7.7 Functional Requirement Satisfaction

Requirement	Solution
Merchant registration	Merchant API + Storage

Webhook configuration	webhook_url field
Secure APIs	API keys & secrets
Secure webhooks	Signature validation

7.7.8 Important Notes

This design is intentionally simple for the functional stage.

At scale, merchant management would additionally require:

- Distributed configuration storage
- Multi-region replication
- Secret management systems
- Rate limiting
- Merchant isolation
- RBAC and permissions

These aspects will be addressed in the scalable architecture section later.

7.8 Customers Are Able to Pay for Merchant's Goods/Services

The primary responsibility of the Payment Gateway is to securely move money from the customer to the merchant.

This requires multiple coordinated steps involving:

- Customer
- Merchant
- Payment API
- Card Network

- Banks
- Internal storage systems

The payment lifecycle consists of:

- Payment creation
- Authorization
- Capture
- Settlement
- Payout

7.8.1 End-to-End Payment Flow

The complete payment flow is executed in the following sequence.

Step 1 — Customer Initiates Payment

The customer reaches the payment page and expresses intent to purchase a product/service.

The merchant frontend calls the Payment API to create a payment entity.

Example request:

None

POST /payment

Example payload:

JSON

```
{  
  
  "merchant_id": "merchant_123",  
  
  "good_id": "product_456",  
  
  "amount": 100,  
}
```

```
"currency": "USD"
```

```
}
```

Purpose:

- Reserve a payment record
- Generate payment identifier
- Enable deduplication later

Step 2 — Create Payment Entity

Payment API creates a new payment record in Payment Storage.

Initial status:

None

NEW

Stored information:

- payment_id
- merchant_id
- amount
- currency
- good_id
- status

Example:

JSON

```
{
```

```
  "payment_id": "pay_abc",
```

```
  "status": "NEW"
```

```
}
```

Step 3 — Customer Submits Card Details

Customer enters:

- Card number
- CVV
- Expiry date

The client application encrypts card data before sending it to Payment API.

Request:

None

POST /payment/{id}/pay

Important:

- Raw card details should never be stored unencrypted
- PCI compliance is required
- TLS encryption mandatory

7.8.2 Authorization Flow

Step 4 — Send Authorization Request to Card Network

Payment API sends an **Authorization** request to:

- Visa
- Mastercard
- Other card processors

Purpose:

- Verify customer card
- Check balance
- Run fraud detection
- Reserve money

Possible outcomes:

- Approved
- Declined
- Requires additional authentication

Interactive Authentication (3DS / OTP)

Sometimes authorization requires customer interaction:

- OTP verification
- Banking app approval
- 3D Secure challenge

This flow usually works through:

- Redirects
- Temporary sessions
- OAuth-like redirection pattern

Example:

None

Customer → Issuing Bank → Confirmation → Redirect back

Step 5 — Store Authorization Result

If authorization succeeds:

- Payment status becomes:

None

AUTHORIZED

Stored data:

- auth_id
- authorization metadata
- timestamps

Why auth_id Matters

auth_id acts as:

- External transaction identifier
- Deduplication mechanism on card networks

This prevents duplicate authorization processing.

7.8.3 Capture Flow

Step 6 — Send Capture Request

After authorization, Payment API sends a **Capture** request.

Purpose:

- Confirm transfer of reserved money
- Finalize transaction

Capture can happen:

- Immediately
- By merchant request
- Via background job

Important property:

- Capture request must be **idempotent**

Step 7 — Update Payment Status

After successful capture:

Payment status becomes:

None

CAPTURED

Actions performed:

- Persist payment state
- Generate ledger entries
- Trigger webhooks
- Prepare payout pipeline

7.8.4 Payout Flow

Step 8 — Transfer Money to Merchant

A background payout job periodically:

- Aggregates captured payments
- Deducts Stripe fees
- Sends payout batch to merchant's bank

Flow:

None

Customer Bank → Stripe Bank → Merchant Bank

Payouts are usually asynchronous.

7.8.5 Additional Operations Performed on Every State Change

During every major step (Step 1 → Step 7), two additional actions occur.

A. Send Webhook Event to Merchant

Whenever payment state changes:

- Merchant receives webhook notification

Examples:

- payment.created
- payment.authorized
- payment.captured
- payment.failed

Delivery guarantees:

- At least once
- Retry on failures

B. Store Event in Audit Storage

Every state transition is persisted in immutable audit storage.

Purpose:

- Financial traceability
- Compliance
- Reconciliation
- Debugging

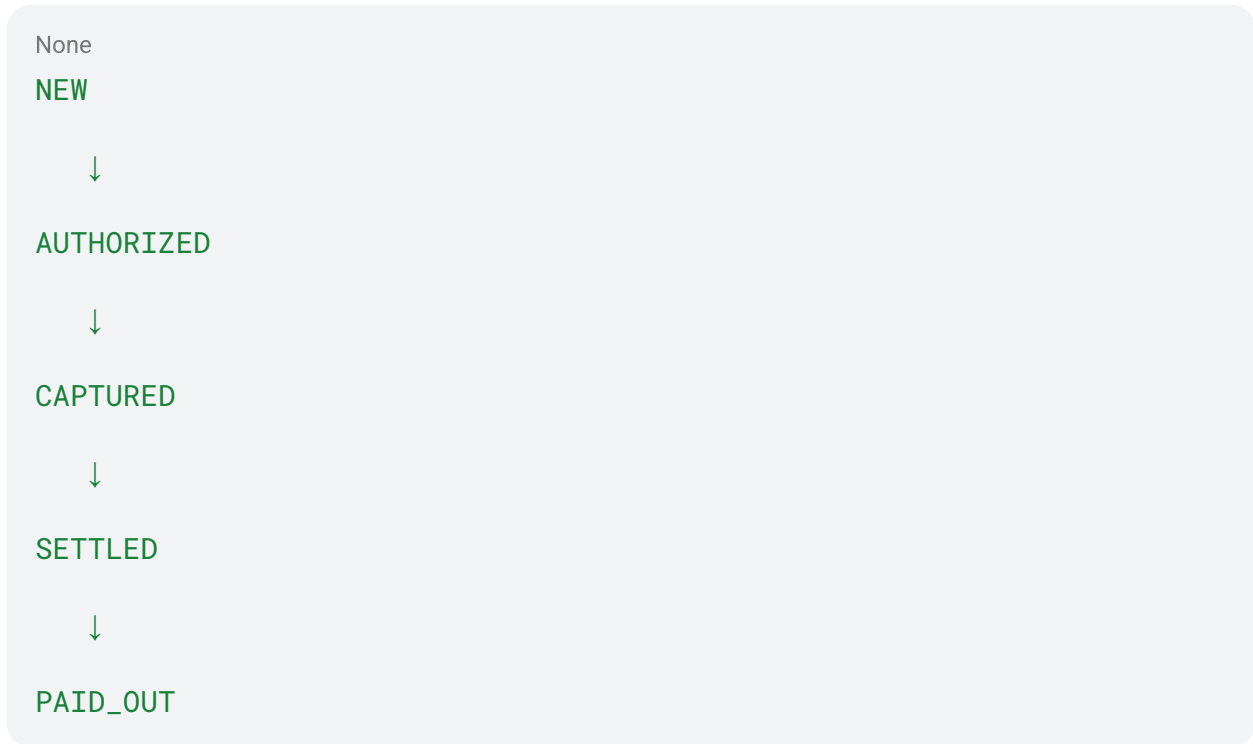
Example event:

JSON

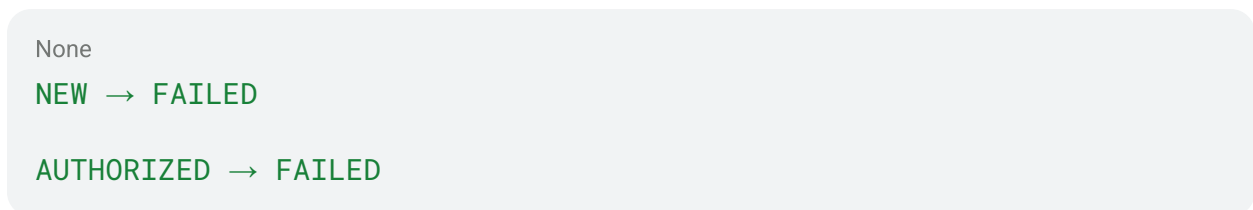
```
{  
  
  "payment_id": "pay_abc",  
  
  "old_status": "AUTHORIZED",  
  
  "new_status": "CAPTURED",  
  
  "timestamp": "..."  
}
```

7.8.6 Payment State Machine

Typical payment lifecycle:



Failure path:



7.8.7 Deduplication & Idempotency

The system uses multiple mechanisms to prevent duplicate charges:

Mechanism	Purpose
payment_id	Internal payment uniqueness
auth_id	Card network deduplication
idempotency keys	Safe retries
immutable ledger	Prevent inconsistent balances

This is critical because:

Double charging is unacceptable in financial systems.

7.8.8 Functional Requirement Satisfaction

Requirement	Solution
Customer payments	Payment API
Secure card handling	Encryption + PCI compliance
Authorization	Card Network integration
Money transfer	Capture flow

Merchant notification	Webhooks
Auditability	Audit Storage
No duplicate payments	Idempotency + auth_id

7.8.9 Important Observations

This simplified design proves:

- End-to-end payment lifecycle works
- Payment state management is clear
- External bank integrations are possible
- Audit trail is maintained
- Webhooks support asynchronous merchant notifications

However, major scalability challenges still remain:

- External network latency
- Synchronous processing bottlenecks
- High write throughput
- Retry handling
- Distributed consistency

These will be addressed in the scalable distributed architecture section.

7.9 Merchant Receives Webhooks About Its Payment

Merchants need to be notified whenever the status of a payment changes.

This is achieved using a **Webhook mechanism**, where the Payment Gateway asynchronously sends HTTP callbacks to the merchant's configured webhook endpoint.

Webhook notifications are triggered on every important payment event or state transition.

7.9.1 Why Webhooks Are Needed

Payments are asynchronous by nature.

For example:

- Authorization may take time
- Capture may happen later
- Settlement may complete hours later
- Payouts happen in background jobs

Instead of merchants constantly polling the API, the payment system proactively pushes updates through webhooks.

7.9.2 Webhook Trigger Points

A webhook is sent whenever:

- Payment is created
- Authorization succeeds/fails
- Capture succeeds/fails
- Refund initiated/completed
- Payout completed
- Chargeback/dispute occurs

Typical events:

Event	Description
payment.created	Payment entity created

payment.authorized	Funds reserved
payment.captured	Payment completed
payment.failed	Payment failure
payout.completed	Merchant payout finished

7.9.3 Webhook Flow

Step 1 — Payment State Changes

Whenever Payment API updates payment status:

None

NEW → AUTHORIZED

AUTHORIZED → CAPTURED

An internal event is generated.

Step 2 — Webhook Event Created

System creates a webhook payload containing:

- Event type
- Payment details
- Metadata
- Timestamp

Example payload:

```
JSON
{
  "event": "payment.captured",
  "payment_id": "pay_123",
  "merchant_id": "merchant_456",
  "amount": 100,
  "currency": "USD",
  "timestamp": "2026-05-11T10:00:00Z"
}
```

Step 3 — Send HTTP Request to Merchant

Webhook service sends HTTP POST request to:

```
None
merchant.webhook_url
```

Example:

```
None
POST https://merchant.com/webhook
```

Step 4 — Merchant Processes Event

Merchant system:

- Verifies webhook signature

- Processes event
- Returns success response

Typical response:

None

HTTP 200 OK

7.9.4 Delivery Guarantees

Webhook systems must provide:

At-Least-Once Delivery

This means:

- Events may be delivered multiple times
- But they should never be silently lost

Reason:

- Network failures are common
- Merchant servers may be temporarily unavailable

Retry Mechanism

If delivery fails:

- Event is retried automatically

Retry triggers:

- Timeout
- 5xx response
- Connection failure

Retries continue for:

None

Up to 30 days

7.9.5 Idempotency Requirement for Merchants

Because events may arrive multiple times:

Merchant webhook handlers must be idempotent.

Meaning:

- Processing the same event twice should not create duplicate side effects.

Merchants usually store:

- event_id
- payment_id

to deduplicate incoming events.

7.9.6 Webhook Security

Webhook endpoints are public-facing, so security is critical.

Signature Validation

Every webhook request contains:

- Cryptographic signature header

Example:

None

X-Stripe-Signature: abc123...

Merchant validates:

- Signature authenticity
- Payload integrity

This prevents:

- Fake webhook requests
- Replay attacks
- Tampering

HTTPS Enforcement

Webhook URLs should:

- Require HTTPS
- Reject insecure endpoints

7.9.7 Internal Storage of Webhook Events

Webhook delivery attempts should also be stored internally.

Stored information:

- event_id
- merchant_id
- payload
- retry_count
- last_attempt_time
- delivery_status

Purpose:

- Reliability
- Debugging

- Auditing
- Replay support

7.9.8 Failure Handling

Possible failures include:

Failure	Handling
Merchant server down	Retry later
Timeout	Retry
Invalid response	Retry
Permanent invalid URL	Disable endpoint after threshold
Duplicate event	Merchant deduplication

7.9.9 Functional Requirement Satisfaction

Requirement	Solution
Merchant notifications	Webhook service

Reliable delivery	Retry mechanism
No silent failures	At-least-once guarantee
Secure callbacks	Signature validation
Auditability	Store webhook delivery logs

7.9.10 Important Design Insight

Webhooks are fundamentally:

Distributed asynchronous event delivery systems.

This introduces challenges such as:

- Retries
- Ordering
- Duplicate delivery
- Backpressure
- Dead-letter handling

At large scale, webhook delivery usually becomes:

- Queue-based
- Distributed
- Rate-limited
- Region-aware

These scalability improvements will be introduced later while satisfying non-functional requirements.

7.10 Merchant Receives Money for Goods/Services (Payout Flow)

After a customer payment is successfully captured and settled, the merchant must receive the money in their bank account.

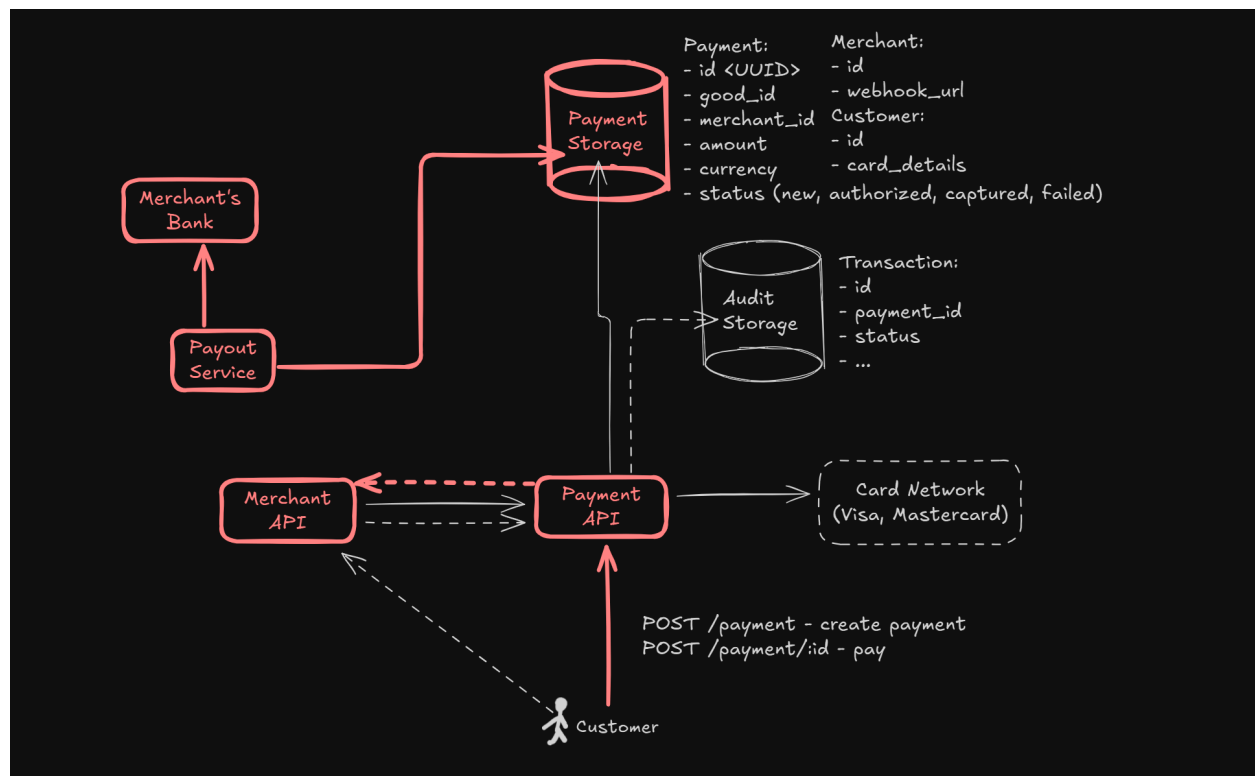
This process is called a **Payout**.

Payouts are usually handled asynchronously through a background service that periodically transfers accumulated merchant balances from Stripe's bank account to the merchant's bank account.

7.10.1 High-Level Payout Architecture

The payout system operates separately from the real-time payment flow.

Payout Flow Diagram



7.10.2 Why Payouts Are Separate

Capturing a payment does not instantly transfer money to the merchant.

There are several reasons:

- Bank settlement may take time
- Fraud/risk checks may still be running
- Refund windows may exist
- Card networks settle in batches
- Payment processors aggregate transfers for efficiency

Therefore:

Merchant payouts are typically asynchronous background operations.

7.10.3 Payout Lifecycle

The payout process generally works as follows.

Step 1 — Successful Payments Collected

The system periodically scans for:

None

CAPTURED / SETTLED payments

Eligible payments are grouped by:

- merchant_id
- currency
- payout schedule

Step 2 — Aggregate Merchant Balance

Payout Service calculates:

None

Total Captured Amount

- Stripe Fees

- Refunds

- Chargebacks

= Final Payout Amount

Example:

Item	Amount
Customer Payments	\$10,000
Stripe Fees	-\$300
Refunds	-\$200
Final Payout	\$9,500

Step 3 — Create Payout Batch

Multiple payments are bundled into a payout batch.

Purpose:

- Reduce banking overhead

- Improve efficiency
- Minimize transfer fees

Example:

```
JSON
{
  "merchant_id": "merchant_123",
  "payments": ["pay_1", "pay_2", "pay_3"],
  "total_amount": 9500
}
```

Step 4 — Transfer Money to Merchant Bank

Payout Service initiates bank transfer:

```
None
Stripe Bank → Merchant Bank
```

Transfer mechanisms may include:

- ACH
- Wire transfer
- SEPA
- RTP/Faster Payments
- UPI/IMPS (region dependent)

Step 5 — Update Internal Ledger

After payout initiation:

- Ledger entries are created

Example:

- Stripe Balance: -\$9500
- Merchant Balance: +\$9500

This maintains:

None

Debit + Credit = 0

7.10.4 Retry Mechanism

Bank transfers can fail due to:

- Network issues
- Bank downtime
- Invalid bank accounts
- Settlement delays

Therefore payout processing must support retries.

Retry Strategy

Failed payouts are:

- Marked as pending retry
- Re-attempted later
- Logged for audit

Typical retry policy:

- Exponential backoff
- Retry limits
- Dead-letter queue for permanent failures

7.10.5 Reconciliation

Since bank systems are asynchronous:

Stripe periodically performs reconciliation between:

- Internal ledger
- Bank settlement records

Purpose:

- Detect missing payouts
- Detect duplicate transfers
- Ensure financial correctness

Example mismatch cases:

Problem	Example
Missing payout	Ledger says paid, bank says failed
Duplicate payout	Retry accidentally succeeded twice
Partial settlement	Bank transferred incorrect amount

Corrections are done via:

- Adjustment ledger entries
- Reprocessing

7.10.6 Payout Scheduling

Merchants may have different payout schedules:

Schedule Type	Example
---------------	---------

Daily	End of each day
Weekly	Every Friday
Instant payout	Real-time transfer
Manual	Merchant-triggered

Configuration stored in merchant profile.

7.10.7 Important Reliability Requirements

Payout systems require extremely high reliability because:

- Real money leaves Stripe accounts
- Errors directly impact merchant trust

Critical guarantees:

- No money loss
- No duplicate payouts
- Full auditability
- Recoverability after failures

7.10.8 Functional Requirement Satisfaction

Requirement	Solution
-------------	----------

Merchant receives money	Payout Service
Batch transfers	Scheduled payout jobs
Fault tolerance	Retry mechanism
Financial correctness	Ledger entries
Consistency with banks	Reconciliation
Auditability	Immutable payout records

7.10.9 Important Design Insight

The payout system is essentially:

A distributed financial settlement engine.

At scale, payout systems require:

- Batch processing pipelines
- Queue-based execution
- Bank adapters
- Distributed reconciliation systems
- Ledger consistency guarantees
- Idempotent payout execution

These scalability improvements will be covered in the distributed architecture section.

7.11 Ensuring No Double Charge for the Same Item/Service

One of the most critical requirements in any payment system is:

A customer must never be charged twice for the same payment.

Double charging can lead to:

- Financial loss
- Customer distrust
- Chargebacks
- Legal and compliance issues

Therefore, payment systems must be designed with strong idempotency and concurrency control mechanisms.

7.11.1 Where Double Charges Can Happen

Since each payment already has a unique `payment_id`, duplicate charging typically occurs due to:

- Client retries
- Network timeouts
- Concurrent requests
- User clicking “Pay” multiple times
- Mobile app retry logic
- Browser refreshes

Example:

None

Customer clicks "Pay" twice rapidly

Without protection:

- Two authorization requests could be sent
- Customer money could be locked twice

7.11.2 Two-Step Card Payment Helps

The payment lifecycle already uses:

1. Authorization
2. Capture

This architecture naturally helps reduce duplicate charging risk because:

- Authorization reserves money first
- Capture finalizes payment later
- Capture is idempotent on card networks

7.11.3 Using Optimistic Concurrency Control

To prevent duplicate authorization requests, the system uses:

Optimistic Concurrency + Compare-And-Swap (CAS)

The idea:

- Only one request should successfully transition payment state
- All concurrent requests must fail safely

7.11.4 Payment State Transition

Initial payment state:

None
NEW

When customer clicks:

None

POST /payment/{id}

The server attempts to atomically update payment status.

7.11.5 Atomic Compare-And-Swap Update

The system executes:

SQL

UPDATE payments

SET status = 'authorizing'

WHERE id = \$id

AND status = 'new';

Why This Works

The database guarantees atomicity.

Only one request can successfully execute:

None

NEW → AUTHORIZING

All concurrent requests will fail because:

- status is no longer NEW

7.11.6 Concurrent Request Scenario

Example:

Two threads attempt payment simultaneously.

Thread	Reads Status	Update Result
Thread A	NEW	Success
Thread B	NEW	Fails (0 rows affected)

Only Thread A proceeds with authorization.

This prevents:

None

Duplicate AUTH requests to Card Network

7.11.7 Authorization Request

Only after successful CAS update:

None

rows_affected > 0

the system sends:

None

AUTHORIZATION request → Card Network

This guarantees:

- Single money reservation
- Single authorization attempt

7.11.8 Persisting Authorization Result

After receiving successful response from Card Network:

```
SQL
UPDATE payments

SET status = 'authorized',

    auth_id = $auth_id

WHERE id = $id

AND status = 'authorizing';
```

This ensures valid state transition:

```
None
AUTHORIZING → AUTHORIZED
```

7.11.9 Why auth_id Is Important

`auth_id` is generated by:

- Card Network
- Issuing Bank

Purpose:

- External deduplication identifier
- Idempotency key on payment networks

If retries happen later:

- Same `auth_id` is reused safely

7.11.10 Capture Is Idempotent

Card networks treat capture requests as idempotent.

Meaning:

- Multiple capture requests for same `auth_id`
- Result in only one actual capture

Example:

None

```
capture(auth_id=123)
```

```
capture(auth_id=123)
```

```
capture(auth_id=123)
```

Outcome:

None

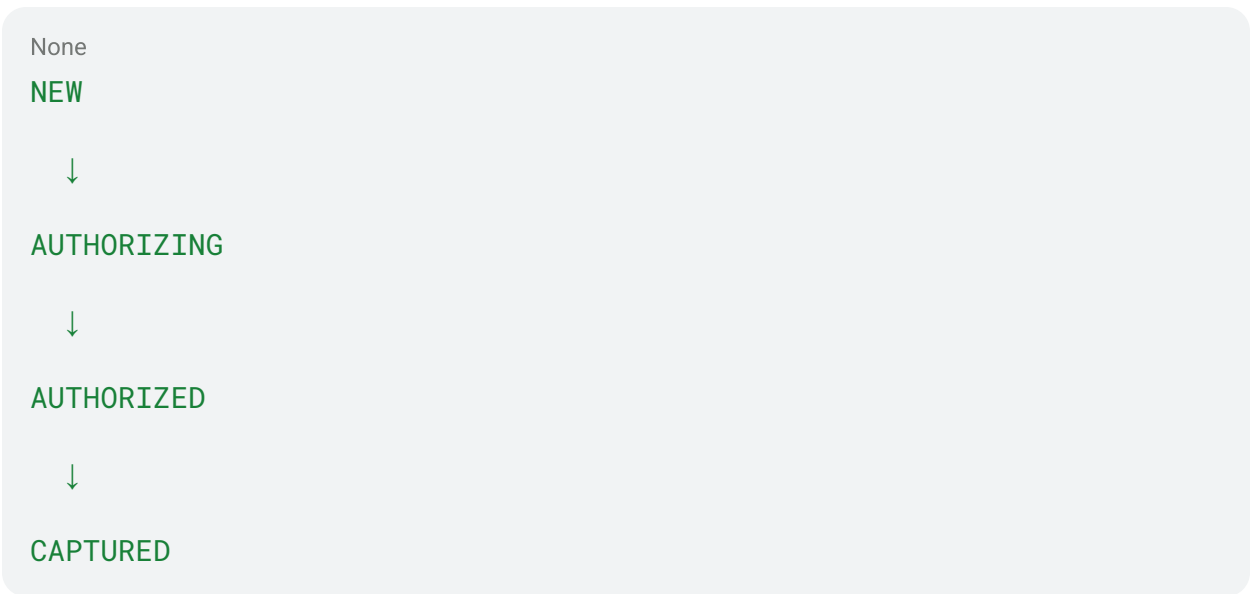
Money captured only once

This is extremely important because:

- Network retries are unavoidable
- Distributed systems are failure-prone

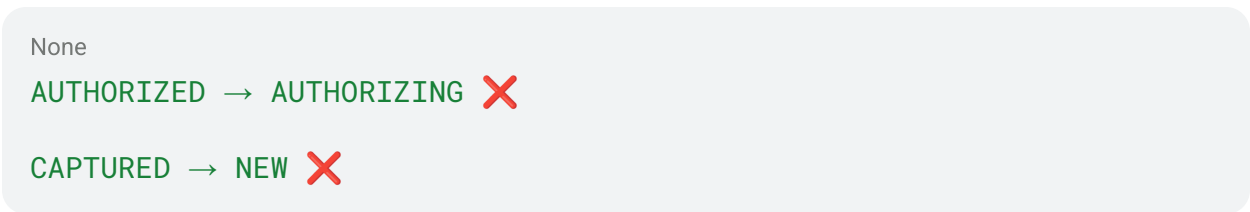
7.11.11 Final Payment State Machine

Valid state transitions:



Invalid transitions are rejected.

Example:



7.11.12 Why Optimistic Concurrency Is Preferred

Compared to database locks:

Optimistic Concurrency	Pessimistic Locking
Faster	Slower

Scales better	Lock contention
No long-held locks	Risk of deadlocks
Good for retries	Higher DB overhead

Payment systems experience:

- Heavy retries
- High concurrency

Therefore CAS-based updates are preferred.

7.11.13 Functional Requirement Satisfaction

Requirement	Solution
No double charge	CAS + state transitions
Safe retries	Idempotent APIs
Prevent concurrent auth	Atomic status updates
Safe capture retries	auth_id idempotency

Correct state machine	Controlled transitions
-----------------------	------------------------

7.11.14 Important Design Insight

Preventing double charges is fundamentally:

A distributed concurrency control problem.

At scale, this additionally requires:

- Idempotency keys
- Distributed locks (rarely)
- Exactly-once semantics approximation
- Retry-safe APIs
- Immutable transaction records

These concepts become even more important in the distributed architecture design later.

7.12 Audit — All Transactions Are Stored in Ledger

A payment system must maintain a complete and immutable financial history of all operations. This is achieved through a **Ledger (Audit Storage)** system.

The ledger acts as:

- Financial source of truth
- Audit trail
- Compliance mechanism
- Reconciliation foundation

Every payment update or event generates a corresponding ledger/audit entry.

7.12.1 Why Ledger Is Required

Financial systems are legally required to maintain:

- Traceable money movement history
- Immutable transaction records
- Historical state transitions

Without a ledger:

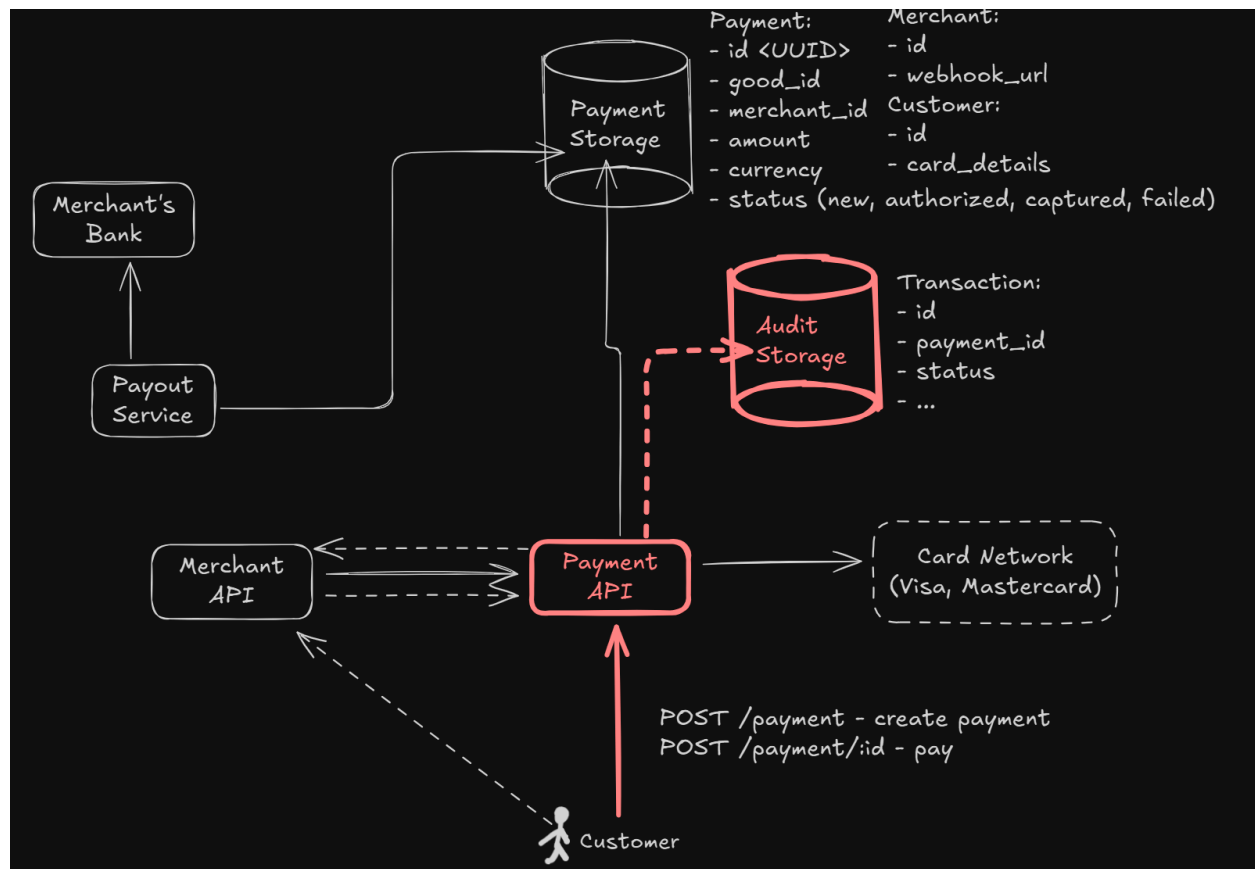
- Reconciliation becomes impossible
- Fraud investigation becomes difficult
- Financial inconsistencies cannot be resolved
- Compliance requirements fail

7.12.2 Ledger Integration in Payment Flow

Every important payment event triggers:

1. Payment state update
2. Ledger entry creation

Audit Storage Integration



7.12.3 What Gets Stored in Ledger

Every financial or payment-related event is recorded.

Examples:

- Payment created
- Authorization successful
- Authorization failed
- Payment captured
- Refund initiated
- Payout completed
- Chargeback created

7.12.4 Example Ledger Entry

JSON

```
{  
  "transaction_id": "txn_123",  
  "payment_id": "pay_456",  
  "event_type": "payment.captured",  
  "amount": 100,  
  "currency": "USD",  
  "timestamp": "2026-05-11T10:00:00Z"  
}
```

7.12.5 Double-Entry Accounting

Ledger systems generally use:

Double-entry accounting

Every movement of money has:

- Debit entry
- Credit entry

Example:

Account	Change
Customer Balance	-\$100

Stripe Holding Account	+\$100
------------------------	--------

After fees:

Account	Change
Stripe Revenue Account	+\$5
Merchant Balance	+\$95

Important invariant:

None

Total Debit + Total Credit = 0

This ensures:

- No money creation
- No money disappearance
- Financial consistency

7.12.6 Append-Only Design

Ledger entries should never be updated or deleted.

Instead:

- New correction entries are appended

Example:

Incorrect:

None

UPDATE ledger SET amount = ...

Correct:

None

INSERT adjustment entry

Benefits:

- Full historical traceability
- Easier audits
- Safer recovery
- Better fraud investigation

7.12.7 Event-Based Ledger Recording

Every payment state transition creates an immutable event.

Example state flow:

None

NEW

↓

AUTHORIZED

↓

CAPTURED

↓

SETTLED

Ledger stores:

- Old state
- New state
- Timestamp
- Actor/system metadata

7.12.8 Audit Storage Requirements

The ledger system has stricter guarantees than regular application databases.

Key requirements:

Requirement	Reason
No data loss	Financial correctness
Immutable records	Legal compliance
Strong durability	Recovery after failures
High write throughput	Millions of events/day
Ordered events	Easier reconciliation

7.12.9 Relationship With Reconciliation

The ledger is later used for:

- Bank reconciliation

- Payout verification
- Chargeback resolution
- Financial reporting

Stripe compares:

None

Ledger Records

vs

Bank Settlement Records

to detect mismatches.

7.12.10 Failure Handling

Even if downstream systems fail:

- Ledger writes should still succeed

Why?

Losing audit data is worse than temporary payment delays.

Typical strategies:

- Write-ahead logging
- Replication
- Append-only storage
- Durable queues

7.12.11 Functional Requirement Satisfaction

Requirement	Solution
Auditability	Immutable ledger
Financial traceability	Transaction history
Compliance	Append-only storage
Reconciliation support	Ledger vs bank matching
Correctness	Double-entry accounting

7.12.12 Important Design Insight

The ledger is arguably:

The most critical component in the entire payment system.

Because:

- APIs can be retried
- Webhooks can be replayed
- Services can fail

But:

None

Ledger correctness must never fail

At scale, ledger systems become:

- Distributed
- Partitioned
- Strongly durable
- Event-driven
- Multi-region replicated

These advanced scalability considerations will be discussed later in the distributed system design section.

7.13 Drawbacks of the Current Architecture

The current single-machine architecture is sufficient for demonstrating the core payment lifecycle and satisfying functional requirements.

However, it only works reliably in:

- Happy-path scenarios
- Low traffic environments
- Minimal failure conditions

Real-world payment systems operate under:

- Massive scale
- Network failures
- Partial system crashes
- Bank inconsistencies
- Long-running retries

The current design has several critical limitations.

7.13.1 Payout Consistency Problems

Problem

Consider the payout flow:

1. Payout Service sends transfer request to merchant's bank

2. Bank successfully processes transfer
3. Before saving payout status internally:
 - Background job crashes

Now:

- Merchant received money
- Internal database thinks payout failed

This creates:

None

Inconsistent financial state

Why This Is Dangerous

Consequences:

- Duplicate payouts during retries
- Incorrect merchant balances
- Failed reconciliation
- Financial losses

Root Cause

External bank systems and internal databases are:

- Separate distributed systems
- Communicating asynchronously

There is no single distributed transaction across:

None

Stripe DB + External Bank

Needed Solution

We need:

- Idempotent payouts
- Reconciliation system
- Reliable event processing
- Durable state transitions

This will be solved later using:

- Distributed workflows
- Retry-safe operations
- Reconciliation pipelines

7.13.2 Webhook Reliability Problems

Problem

What happens if:

None

Merchant webhook endpoint is down for 1 day?

Current design:

- Sends webhook directly
- Failure may lose notification permanently

Consequences

Merchant may never learn:

- Payment succeeded
- Refund happened
- Payout completed

This creates:

- Business inconsistencies
- Order processing failures
- Customer support issues

Needed Solution

Webhook delivery requires:

- Persistent event queues
- Retry mechanisms
- Dead-letter queues
- Delivery tracking

Target guarantee:

None

At-least-once delivery

for up to:

None

30 days of retries

7.13.3 Ledger/Audit Data Loss

Problem

Suppose:

1. Payment API processes transaction
2. Customer charged successfully
3. API crashes before writing to Audit Storage

Now:

- Money moved successfully
- Ledger missing transaction

This is catastrophic because:

None

Financial history becomes incomplete

Why This Is Critical

Ledger systems are legal requirements.

Missing ledger entries mean:

- Compliance violation
- Failed audits
- Broken reconciliation
- Impossible forensic analysis

Needed Solution

Ledger writes must become:

- Durable
- Atomic
- Retry-safe
- Impossible to skip

Typical approaches:

- Event sourcing
- WAL (Write-Ahead Log)
- Transactional outbox pattern
- Durable message queues

7.13.4 Scalability Limitations

Problem

Current system uses:

- Single machine
- Single database
- Synchronous processing

This severely limits throughput.

Estimated Bottlenecks

The system cannot realistically handle:

- 10k+ payment RPS
- 100k+ ledger events/sec
- Millions of merchants

Issues include:

- Database contention
- CPU bottlenecks
- Network bottlenecks
- Blocking external calls

Consequences

Under heavy traffic:

- Increased latency
- Timeouts
- Failed payments
- Queue buildup

Especially during:

None

Black Friday / seasonal spikes

7.13.5 Availability & Fault Tolerance Issues

Problem

Current architecture has:

None

Single points of failure

Examples:

- Payment API crashes
- Database unavailable
- Webhook service dies

Entire payment flow stops.

Why This Is Unacceptable

Payment systems require:

- Extremely high uptime
- Graceful degradation
- Fast recovery

Downtime directly impacts:

- Revenue
- Merchant trust
- Customer experience

Needed Solution

We need:

- Replication
- Horizontal scaling

- Load balancing
- Distributed queues
- Multi-region deployment
- Failover mechanisms

7.13.6 Tight Coupling & Synchronous Processing

Current design tightly couples:

- Payments
- Webhooks
- Audit writes
- External network calls

As a result:

- One slow component blocks others
- Cascading failures occur
- Retry logic becomes complicated

Needed Solution

Introduce:

- Asynchronous event-driven architecture
- Message queues
- Independent workers
- Event streaming

7.13.7 Lack of Durable Workflow Management

Complex flows such as:

- Authorization
- Capture
- Payout
- Refund

span multiple systems and may take minutes or days.

Current design lacks:

- Workflow orchestration
- Retry tracking
- State recovery

Needed Solution

Use:

- Durable workflow engines
- State machines
- Saga pattern
- Persistent job execution

7.13.8 Summary of Current Drawbacks

Problem	Impact
Payout inconsistency	Duplicate/lost payouts
Webhook failures	Merchant misses events
Missing ledger writes	Compliance violation

Single DB bottleneck	Poor scalability
Single points of failure	Low availability
Tight coupling	Cascading failures
No workflow durability	Hard recovery

7.13.9 Transition to Distributed Architecture

The current architecture successfully demonstrates:

- Payment correctness
- Functional flows
- Basic state management

However, it is not production-grade.

The next stage of the design focuses on:

- Scalability
- High availability
- Distributed consistency
- Fault tolerance
- Durable event processing
- Reliable ledger guarantees

All these concerns will be addressed in the distributed system architecture section that follows.